| | |
|---|---|
| **Status** | Finished |
| **Started** | Monday, 31 August 2026, 10:56 AM |
| **Completed** | Monday, 31 August 2026, 11:16 AM |
| **Duration** | 19 mins 7 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

Question **1**

Correct

Mark 1.00 out of 1.00

The Chinese zodiac assigns animals to years in a 12 year cycle. One 12 year cycle is shown in the table below. The pattern repeats from there, with 2012 being another year of the dragon, and 1999 being another year of the hare.

Year Animal

2000 Dragon

2001 Snake

2002 Horse

2003 Sheep

2004 Monkey

2005 Rooster

2006 Dog

2007 Pig

2008 Rat

2009 Ox

2010 Tiger

2011 Hare

Write a program that reads a year from the user and displays the animal associated with that year. Your program should work correctly for any year greater than or equal to zero, not just the ones listed in the table.

Sample Input 1

2010

Sample Output 1

2010 is the year of the Tiger.

Sample Input 2

2020

Sample Output 2

2020 is the year of the Rat.

**Answer:** (penalty regime: 0 %)

```
year = int(input())

animals = [
    "Monkey",
    "Rooster",
    "Dog",
    "Pig",
    "Rat",
    "Ox",
    "Tiger",
    "Hare",
    "Dragon",
    "Snake",
    "Horse",
    "Sheep",
]


index = year % 12

animal = animals[index]

print(f"{year} is the year of the {animal}.")

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 2010 | 2010 is the year of the Tiger. | 2010 is the year of the Tiger. | ✓ |
| ✓ | 2020 | 2020 is the year of the Rat. | 2020 is the year of the Rat. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Most years have 365 days. However, the time required for the Earth to orbit the Sun is actually slightly more than that. As a result, an extra day, February 29, is included in some years to correct for this difference. Such years are referred to as leap years. The rules for determining whether or not a year is a leap year follow:

• Any year that is divisible by 400 is a leap year.

• Of the remaining years, any year that is divisible by 100 is not a leap year.

• Of the remaining years, any year that is divisible by 4 is a leap year.

• All other years are not leap years.

Write a program that reads a year from the user and displays a message indicating whether or not it is a leap year.

Sample Input 1

1900

Sample Output 1

1900 is not a leap year.

Sample Input 2

2000

Sample Output 2

2000 is a leap year.

**Answer:** (penalty regime: 0 %)

```
year = int(input())
if year % 400 == 0:
    print(f"{year} is a leap year.")
elif year % 100 == 0:
    print(f"{year} is not a leap year.")
elif year % 4 == 0:
    print(f"{year} is a leap year.")
else:
    print(f"{year} is not a leap year.")

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 1900 | 1900 is not a leap year. | 1900 is not a leap year. | ✓ |
| ✓ | 2000 | 2000 is a leap year. | 2000 is a leap year. | ✓ |
| ✓ | 2100 | 2100 is not a leap year. | 2100 is not a leap year. | ✓ |
| ✓ | 2020 | 2020 is a leap year. | 2020 is a leap year. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

A triangle can be classified based on the lengths of its sides as equilateral, isosceles or scalene. All three sides of an equilateral triangle have the same length. An isosceles triangle has two sides that are the same length, and a third side that is a different length. If all of the sides have different lengths then the triangle is scalene.

Write a program that reads the lengths of the three sides of a triangle from the user. Then display a message that states the triangle's type.

Sample Input 1

60

60

60

Sample Output 1

That's a equilateral triangle

Sample Input 2

40

40

80

Sample Output 2

That's a isosceles triangle

Sample Input 3

50

60

70

Sample Output 3

That's a scalene triangle

**For example:**

| Input | Result |
|---|---|
| 60<br>60<br>60 | That's a equilateral triangle |
| 40<br>40<br>80 | That's a isosceles triangle |

**Answer:** (penalty regime: 0 %)

```
side1 = float(input())
side2 = float(input())
side3 = float(input())
if side1 == side2 == side3:
  print("That's a equilateral triangle")
elif side1 == side2 or side2 == side3 or side1 == side3:
  print("That's a isosceles triangle")
else:
  print("That's a scalene triangle")

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 60<br>60<br>60 | That's a equilateral triangle | That's a equilateral triangle | ✓ |
| ✓ | 40<br>40<br>80 | That's a isosceles triangle | That's a isosceles triangle | ✓ |
| ✓ | 50<br>60<br>70 | That's a scalene triangle | That's a scalene triangle | ✓ |
| ✓ | 50<br>50<br>80 | That's a isosceles triangle | That's a isosceles triangle | ✓ |
| ✓ | 10<br>10<br>10 | That's a equilateral triangle | That's a equilateral triangle | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

IN / OUT

Ms. Sita, the faculty handling programming lab for you is very strict. Your seniors have told you that she will not allow you to enter the week's lab if you have not completed atleast half the number of problems given last week. Many of you didn't understand this statement and so they requested the good programmers from your batch to write a program to find whether a student will be allowed into a week's lab given the number of problems given last week and the number of problems solved by the student in that week.

Input Format:

Input consists of 2 integers.

The first integer corresponds to the number of problems given and the second integer corresponds to the number of problems solved.

Output Format:

Output consists of the string "IN" or "OUT".

Sample Input and Output:

Input

8

3

Output

OUT

**For example:**

| Input | Result |
|---|---|
| 8<br>3 | OUT |

**Answer:** (penalty regime: 0 %)

```
import math

def check_lab_entry():
    total_problems = int(input())
    solved_problems = int(input())

    required_problems = math.ceil(total_problems / 2)

    if solved_problems >= required_problems:
        print("IN")
    else:
        print("OUT")

if __name__ == "__main__":
    check_lab_entry()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 8<br>3 | OUT | OUT | ✓ |
| ✓ | 8<br>5 | IN | IN | ✓ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 20<br>9 | OUT | OUT | ✓ |
| ✓ | 50<br>31 | IN | IN | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a program to calculate and print the Electricity bill where the unit consumed by the user is given from test case. It prints the total amount the customer has to pay. The charge are as follows:

| Unit | Charge / Unit |
|---|---|
| Upto 199 | @1.20 |
| 200 and above but less than 400 | @1.50 |
| 400 and above but less than 600 | @1.80 |
| 600 and above | @2.00 |

If bill exceeds Rs.400 then a surcharge of 15% will be charged and the minimum bill should be of Rs.100/-

Sample Test Cases

Test Case 1

Input

50

Output

100.00

Test Case 2

Input

300

Output

517.50

**For example:**

| Input | Result |
|---|---|
| 100.00 | 120.00 |
| 500 | 1035.00 |

**Answer:** (penalty regime: 0 %)

```
def calculate_electricity_bill(units):
    if units < 200:
        rate = 1.20
    elif units < 400:
        rate = 1.50
    elif units < 600:
        rate = 1.80
    else:
        rate = 2.00

    bill_amount = units * rate

    if bill_amount > 400:
        bill_amount += bill_amount * 0.15

    if bill_amount < 100:
        bill_amount = 100.00

    return bill_amount

try:
    user_units = float(input())
    total_bill = calculate_electricity_bill(user_units)
    print(f"{total_bill:.2f}")
except ValueError:
    print("Please enter a valid numeric value for units.")

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 50 | 100.00 | 100.00 | ✓ |
| ✓ | 100.00 | 120.00 | 120.00 | ✓ |
| ✓ | 500 | 1035.00 | 1035.00 | ✓ |
| ✓ | 700 | 1610.00 | 1610.00 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

A certain type of steel is used to test and give grade according to the following conditions.

1. Hardness of the steel must be greater than 50
2. Carbon content of the steel must be less than 0.7
3. Tensile strength must be greater than 5600

The grades awarded are as follows:

- Grade is 10 if all three conditions are met
- Grade is 9 if conditions (1) and (2) are met
- Grade is 8 if conditions (2) and (3) are met
- Grade is 7 if conditions (1) and (3) are met
- Grade is 6 if only one condition is met
- Grade is 5 if none of the three conditions are met

Write a program to display the grade of the steel, based on the values of hardness, carbon content and tensile strength of the steel, given by the user.

Input

53

0.6

5602

Output:

10

**Answer:** (penalty regime: 0 %)

```
hardness = float(input())
carbon = float(input())
tensile = float(input())

cond1 = hardness > 50
cond2 = carbon < 0.7
cond3 = tensile > 5600

if cond1 and cond2 and cond3:
  grade = 10
elif cond1 and cond2:
  grade = 9
elif cond2 and cond3:
  grade = 8
elif cond1 and cond3:
  grade = 7
elif cond1 or cond2 or cond3:
  grade = 6
else:
  grade = 5

print(grade)

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 53<br>0.6<br>5602 | 10 | 10 | ✓ |
| ✓ | 45<br>0<br>4500 | 6 | 6 | ✓ |

Passed all tests! ✓

Correct

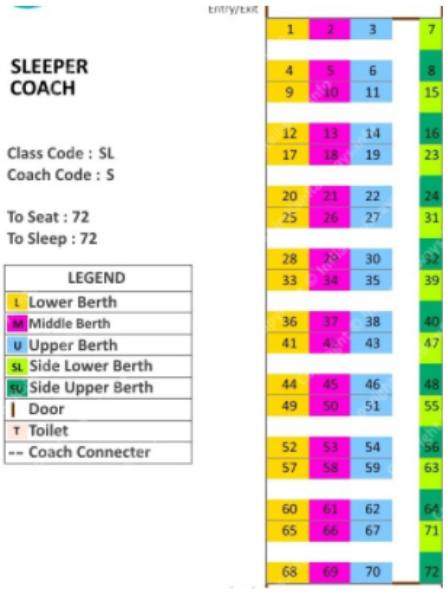
Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a program to determine the type of berth when the seat / berth number in the train is given.




**Input Format:**

Input consists of a single integer. Assume that the range of input is between 1 and 72.

**Output Format:**

Output consists of a single string. [Upper or Middle or Lower or Side Lower or Side Upper]

**Sample Input 1:**

9

**Sample Output 1:**

Lower Berth

**Answer:** (penalty regime: 0 %)

```
def determine_berth(seat_no):
    if seat_no < 1 or seat_no > 72:
        return "Invalid Seat Number"

    remainder = seat_no % 8

    if remainder == 1 or remainder == 4:
        return "Lower Berth"
    elif remainder == 2 or remainder == 5:
        return "Middle Berth"
    elif remainder == 3 or remainder == 6:
        return "Upper Berth"
    elif remainder == 7:
        return "Side Lower Berth"
    elif remainder == 0:
        return "Side Upper Berth"

try:
    seat_number = int(input())
    print(determine_berth(seat_number))
except ValueError:
    print("Please enter a valid integer")

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 9 | Lower Berth | Lower Berth | ✓ |
| ✓ | 72 | Side Upper Berth | Side Upper Berth | ✓ |
| ✓ | 20 | Lower Berth | Lower Berth | ✓ |
| ✓ | 34 | Middle Berth | Middle Berth | ✓ |
| ✓ | 39 | Side Lower Berth | Side Lower Berth | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Write a Python program that accepts three parameters. The first parameter is an integer. The second is one of the following mathematical operators: +, -, /, or *. The third parameter will also be an integer.

The function should perform a calculation and return the results. For example, if the function is passed 6 and 4, it should return 24.

Sample Input Format:

11

+

14

Sample Output  Format:

25

**Answer:** (penalty regime: 0 %)

```
def calculate(num1, operator, num2):
    if operator == '+':
        return num1 + num2
    elif operator == '-':
        return num1 - num2
    elif operator == '*':
        return num1 * num2
    elif operator == '/':
        if num2 == 0:
            return "Error: Division by zero"
        return num1 / num2
    else:
        return "Error: Invalid operator"

try:
    val1 = int(input().strip())
    op = input().strip()
    val2 = int(input().strip())

    result = calculate(val1, op, val2)
    print(result)
except ValueError:
    print("Error: Please enter valid integers")

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 11<br>+<br>14 | 25 | 25 | ✓ |
| ✓ | 45<br>-<br>50 | -5 | -5 | ✓ |
| ✓ | 12<br>*<br>100 | 1200 | 1200 | ✓ |
| ✓ | 18<br>/<br>2 | 9.0 | 9.0 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a program that accepts 5 inputs and returns the count of how many of those 5 are odd.

For example,

If the five inputs are 12, 17, 19, 14, and 115, there are three odd numbers 17, 19 and 115. So, the program must return 3.

Similarly,

If the five inputs are 15, 0, -12, 19, and 28, there are two odd numbers 15 and 19. So, the program must return 2.

Observe that zero is considered an even number.

**For example:**

| Input | Result |
|---|---|
| 12<br>17<br>19<br>14<br>115 | 3 |
| 15<br>0<br>-12<br>19<br>28 | 2 |

**Answer:** (penalty regime: 0 %)

```
count = 0

for i in range(5):
    n = int(input())
    if n % 2 != 0:
        count += 1

print(count)

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 12<br>17<br>19<br>14<br>115 | 3 | 3 | ✓ |
| ✓ | 15<br>0<br>-12<br>19<br>28 | 2 | 2 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given an integer N, check whether N the given number can be made a perfect square after adding 1 to it.

Input Format:

Single integer input.

Output Format:

Yes or No.

Example Input:

24

Output:

Yes

Example Input:

26

Output:

No

**For example:**

| Input | Result |
|---|---|
| 24 | Yes |

**Answer:** (penalty regime: 0 %)

```
n = int(input())

x = n + 1

if int(x ** 0.5) ** 2 == x:
    print("Yes")
else:
    print("No")

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 24 | Yes | Yes | ✓ |
| ✓ | 26 | No | No | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.